Problem List | Submit | 0 | 359:00:56 | Premium

Description | Accepted | Editorial | Solutions | Submissions

876. Middle of the Linked List

Solved

Easy | Topics | Companies

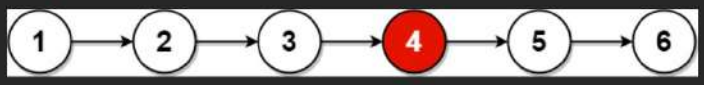
Given the `head` of a singly linked list, return *the middle node of the linked list.*

If there are two middle nodes, return **the second middle** node.

Example 1:

```
Input: head = [1,2,3,4,5]
Output: [3,4,5]
Explanation: The middle node of the list is node 3.
```

Example 2:




```
Input: head = [1,2,3,4,5,6]
Output: [4,5,6]
Explanation: Since the list has two middle nodes with values 3 and 4, we return
the second one
```

13.2K | 261 | 105 Online

Code

Java | Auto

```java
class Solution {
    public ListNode middleNode(ListNode head) {
        ListNode slow = head;
        ListNode fast = head;

        while (fast != null && fast.next != null) {
            slow = slow.next;        // move 1 step
            fast = fast.next.next;   // move 2 steps
        }

        return slow;
    }
}
```

Saved | Ln 13, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

head =
[1,2,3,4,5]

Output

Problem List | Submit | 0 | 359:00:56 | Premium

Description | Accepted | Editorial | Solutions | Submissions

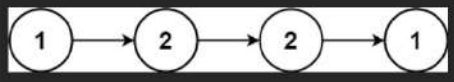
234. Palindrome Linked List

Solved

Easy | Topics | Companies

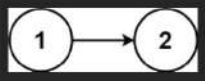
Given the `head` of a singly linked list, return `true` *if it is a palindrome or* `false` *otherwise.*

Example 1:




```
Input: head = [1,2,2,1]
Output: true
```

Example 2:




```
Input: head = [1,2]
Output: false
```

Constraints:

18.3K | 392 | 91 Online

Code

Java | Auto

```java
    // Helper function to reverse list
    private ListNode reverse(ListNode head) {
        ListNode prev = null;

        while (head != null) {
            ListNode next = head.next;
            head.next = prev;
            prev = head;
            head = next;
        }

        return prev;
    }
}
```

Saved | Ln 45, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

head =
[1,2,2,1]

Output

https://leetcode.com/problems/merge-two-sorted-lists/description/

Problem List | Submit | 0 | 359:00:56 | Premium

Description | Accepted | Editorial | Solutions | Submissions

21. Merge Two Sorted Lists

Solved

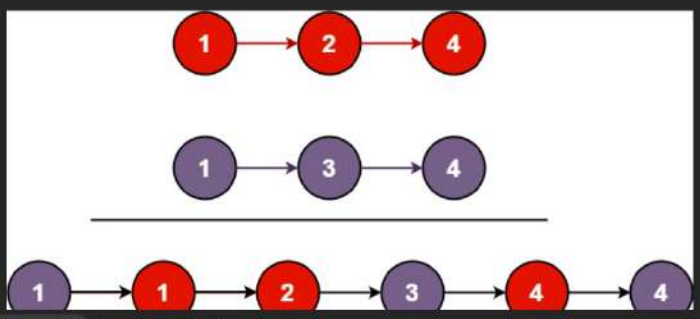
Easy | Topics | Companies

You are given the heads of two sorted linked lists `list1` and `list2`.

Merge the two lists into one **sorted** list. The list should be made by splicing together the nodes of the first two lists.

Return *the head of the merged linked list.*

Example 1:



25K | 596 | 302 Online

Code

Java | Auto

```java
class Solution {
    public ListNode mergeTwoLists(ListNode list1, ListNode list2) {

        // Dummy node to start the merged list
        ListNode dummy = new ListNode(-1);
        ListNode current = dummy;

        // Traverse both lists
        while (list1 != null && list2 != null) {
            if (list1.val <= list2.val) {
                current.next = list1;
                list1 = list1.next;
            } else {
                current.next = list2;
```

Saved | Ln 29, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

list1 =
[1,2,4]

list2 =

https://leetcode.com/problems/merge-k-sorted-lists/description/

Problem List | Submit | 0 | 359:00:56 | Premium

Description | Accepted | Editorial | Solutions | Submissions

23. Merge k Sorted Lists

Solved

Hard | Topics | Companies

You are given an array of `k` linked-lists `lists`, each linked-list is sorted in ascending order.

Merge all the linked-lists into one sorted linked-list and return it.

Example 1:

```
Input: lists = [[1,4,5],[1,3,4],[2,6]]
Output: [1,1,2,3,4,4,5,6]
Explanation: The linked-lists are:
[
  1->4->5,
  1->3->4,
  2->6
]
merging them into one sorted linked list:
1->1->2->3->4->4->5->6
```

Example 2:

```
Input: lists = []
Output: []
```

Example 3:

21.3K | 405 | 231 Online

Code

Java | Auto

```java
import java.util.PriorityQueue;

class Solution {
    public ListNode mergeKLists(ListNode[] lists) {

        // Min Heap based on node value
        PriorityQueue<ListNode> pq = new PriorityQueue<>(
            (a, b) -> a.val - b.val
        );

        // Add all list heads
        for (ListNode node : lists) {
            if (node != null) {
                pq.add(node);
            }
```

Saved | Ln 35, Col 2

Testcase | Test Result

Accepted Runtime: 1 ms

Case 1 | Case 2 | Case 3

Input

lists =

[[1,4,5],[1,3,4],[2,6]]

Output

https://leetcode.com/problems/reverse-linked-list/description/

Problem List | Submit | 359:00:56 | Premium

Description | Accepted | Editorial | Solutions | Submissions

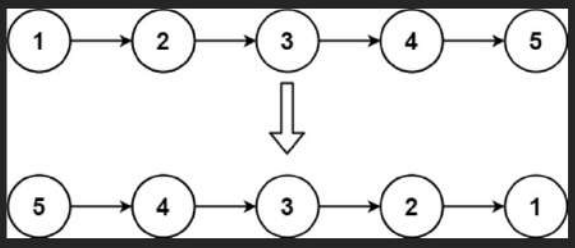
206. Reverse Linked List

Solved

Easy | Topics | Companies

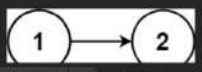
Given the `head` of a singly linked list, reverse the list, and return *the reversed list*.

Example 1:




```
Input: head = [1,2,3,4,5]
Output: [5,4,3,2,1]
```

Example 2:



24.5K | 401 | 252 Online

Code

Java | Auto

```java
        ListNode prev = null;
        ListNode curr = head;

        while (curr != null) {
            ListNode next = curr.next; // store next
            curr.next = prev;          // reverse link
            prev = curr;               // move prev forward
            curr = next;               // move curr forward
        }

        return prev; // new head
    }
}
```

Saved | Ln 16, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

head =
[1,2,3,4,5]

Output

https://leetcode.com/problems/reverse-nodes-in-k-group/

Problem List | Submit | 0 | 359:00:56 | Premium

Description | Accepted | Editorial | Solutions | Submissions

25. Reverse Nodes in k-Group

Solved

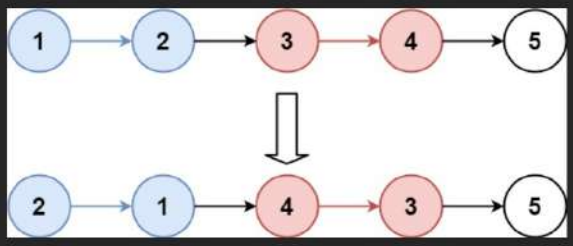
Hard | Topics | Companies

Given the `head` of a linked list, reverse the nodes of the list `k` at a time, and return *the modified list*.

`k` is a positive integer and is less than or equal to the length of the linked list. If the number of nodes is not a multiple of `k` then left-out nodes, in the end, should remain as it is.

You may not alter the values in the list's nodes, only nodes themselves may be changed.

Example 1:




```
Input: head = [1,2,3,4,5], k = 2
Output: [2,1,4,3,5]
```

15.7K | 339 | 101 Online

Code

Java | Auto

```
            while (curr != nextGroupStart) {
                ListNode temp = curr.next;
                curr.next = prev;
                prev = curr;
                curr = temp;
            }

            // Connect previous part
            prevGroupEnd.next = kth;
            prevGroupEnd = groupStart;
        }
    }
}
```

Saved | Ln 37, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

head =
[1,2,3,4,5]

k =

https://leetcode.com/problems/linked-list-cycle-ii/description/

Problem List | Submit | 0 | 359:00:56 | Premium

Description | Accepted | Editorial | Solutions | Submissions

142. Linked List Cycle II

Solved

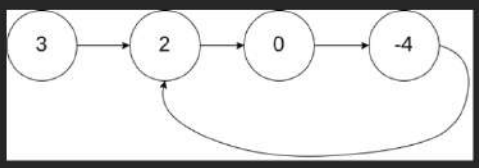
Medium | Topics | Companies

Given the `head` of a linked list, return *the node where the cycle begins. If there is no cycle, return* `null`.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the `next` pointer. Internally, `pos` is used to denote the index of the node that tail's `next` pointer is connected to (**0-indexed**). It is `-1` if there is no cycle. **Note that** `pos` **is not passed as a parameter.**

Do not modify the linked list.

Example 1:




```
Input: head = [3,2,0,-4], pos = 1
Output: tail connects to node index 1
Explanation: There is a cycle in the linked list, where tail connects to the
second node.
```

15.2K | 262 | 70 Online

Code

Java | Auto

```java
            ListNode entry = head;

            while (entry != slow) {
                entry = entry.next;
                slow = slow.next;
            }

            return entry; // cycle start
        }
    }

    return null; // no cycle
  }
}
```

Saved | Ln 30, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

head =
[3,2,0,-4]

pos =
1

https://leetcode.com/problems/intersection-of-two-linked-lists/description/

Problem List | Submit | 0 | 359:00:56 | Premium

Description | Accepted | Editorial | Solutions | Submissions

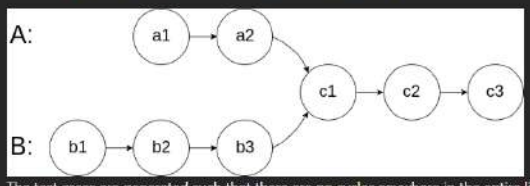
160. Intersection of Two Linked Lists

Solved

Easy | Topics | Companies

Given the heads of two singly linked-lists `headA` and `headB`, return *the node at which the two lists intersect.* If the two linked lists have no intersection at all, return `null`.

For example, the following two linked lists begin to intersect at node `c1`:



The test cases are generated such that there are no cycles anywhere in the entire linked structure.

Note that the linked lists must **retain their original structure** after the function returns.

Custom Judge:

The inputs to the **judge** are given as follows (your program is **not** given these inputs):

- `intersectVal` - The value of the node where the intersection occurs. This is `0` if there is no intersected node.
- `listA` - The first linked list.

16.6K | 291 | 79 Online

Code

Java | Auto

```java
public class Solution {
    public ListNode getIntersectionNode(ListNode headA, ListNode headB) {

        if (headA == null || headB == null) return null;

        ListNode a = headA;
        ListNode b = headB;

        while (a != b) {
            a = (a == null) ? headB : a.next;
            b = (b == null) ? headA : b.next;
        }

        return a; // intersection node or null
    }
```

Saved | Ln 16, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

intersectVal =

8

listA =

[4, 1, 8, 4, 5]